THE UNIVERSITY of TENNESSEE KNOXVILLE

Lecture 22: Multistep decisions

Instructor: Sergei V. Kalinin

From agents to search

- **Given:**
  - An initial state of the world
  - A set of possible actions or operators that can be performed.
  - A goal test that can be applied to a single state of the world to determine if it is a goal state.

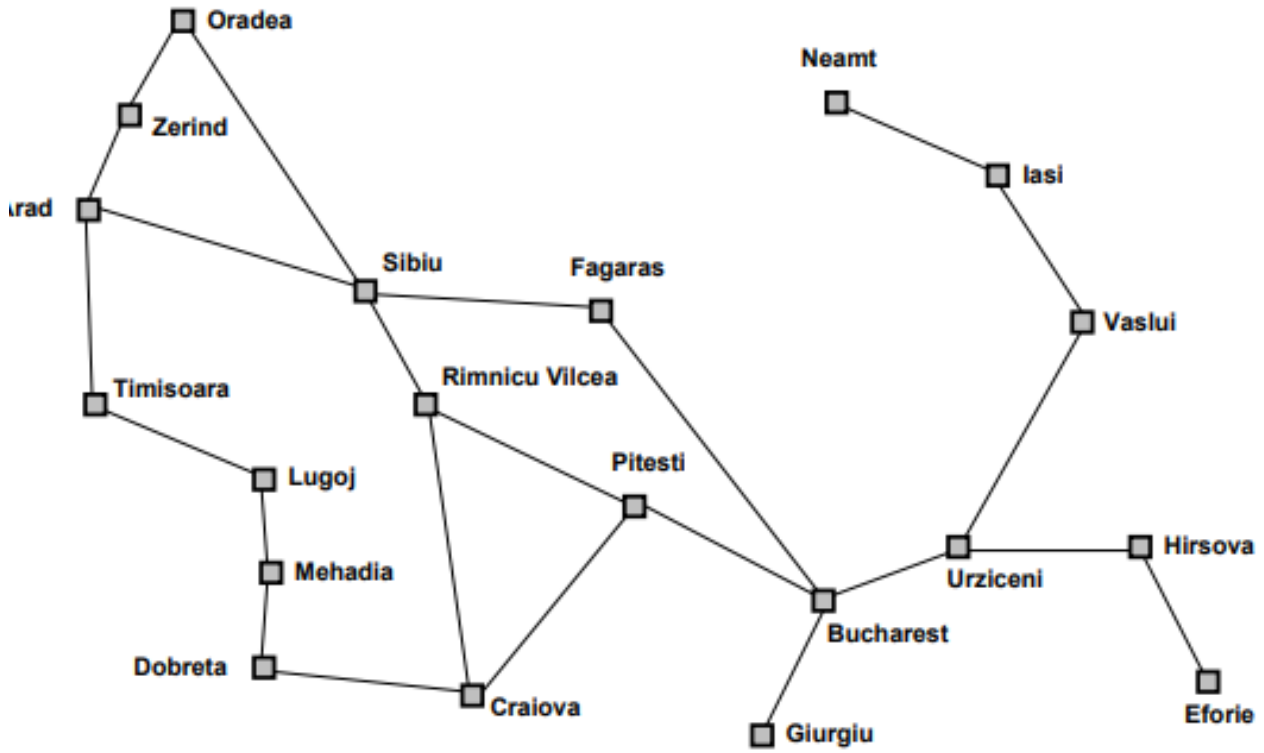
- **Find:** A solution stated as a path of states and operators that shows how to transform the initial state into one that satisfies the goal test.

- The initial state and set of operators implicitly define a state space of states of the world and operator transitions between them. May be infinite.

From agents to search

- Rational agents need to perform sequences of actions to achieve goals
- Intelligent behavior can be generated by having a look-up table or reactive policy that tells the agent what to do in every circumstance, but:
    - Such a table or policy is difficult to build
    - All contingencies must be anticipated
- A more general approach is for the agent to have knowledge of the world and how its actions affect it and be able to simulate execution of actions in an internal model of the world in order to determine a sequence of actions that will accomplish its goals
- This is the general task of problem solving and is typically performed by searching through an internally modelled space of world states

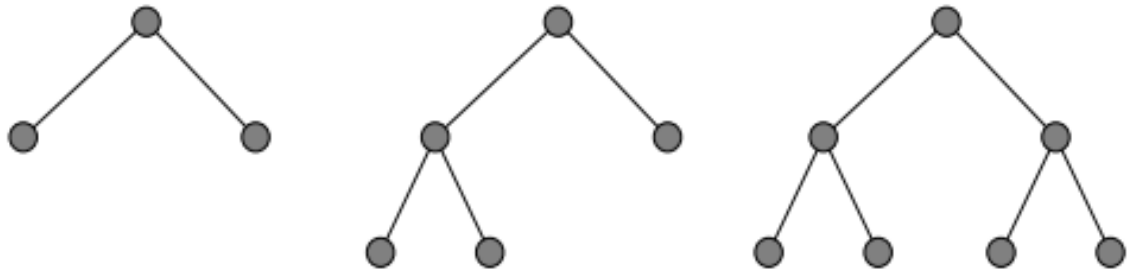
Route finding



- Initial state: Arad
- Goal state: Bucharest
- Path cost: Number of intermediate cities, distance traveled, expected travel tim

Breadth-first search

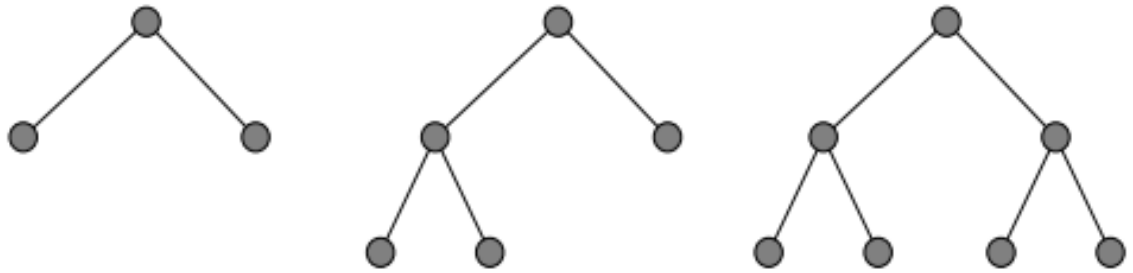
- Expands search nodes level by level, all nodes at level *d* are expanded before expanding nodes at level *d*+1



- Implemented by adding new nodes to the end of the queue
- Since eventually visits every node to a given depth, guaranteed to be complete
- Optimal provided path cost is a nondecreasing function of the depth of the node (e.g. all operators of equal cost) since nodes explored in depth order

Breadth-first search

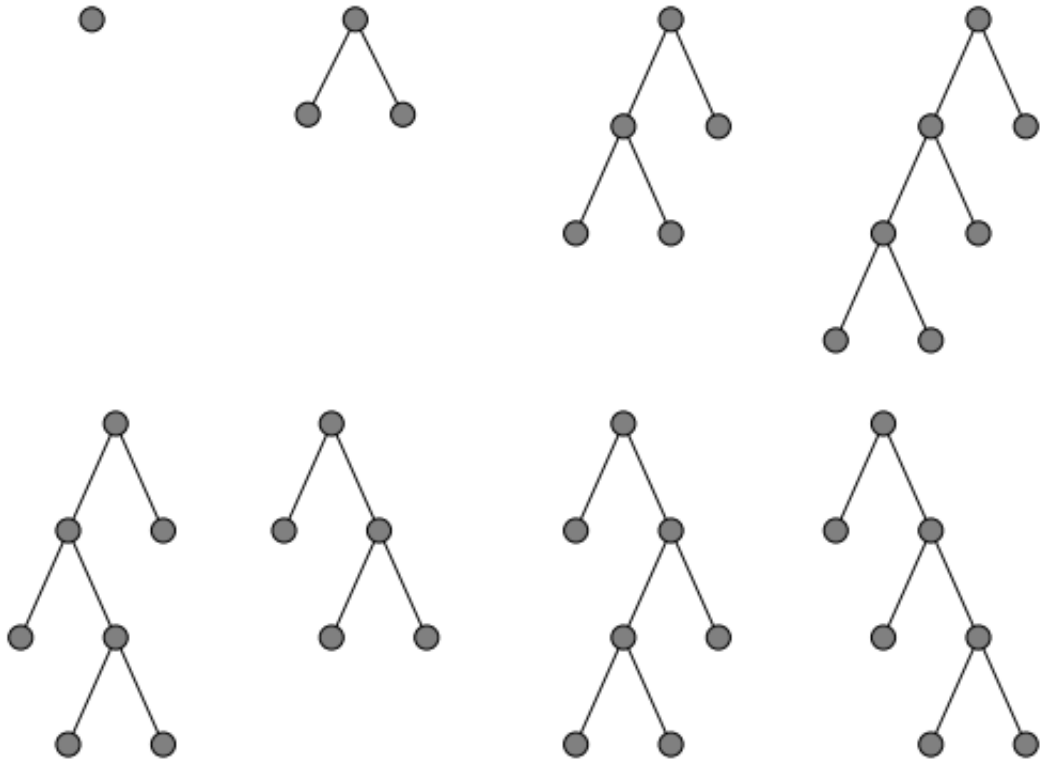
- Expands search nodes level by level, all nodes at level *d* are expanded before expanding nodes at level *d*+1



- Implemented by adding new nodes to the end of the queue
- Since eventually visits every node to a given depth, guaranteed to be complete
- Optimal provided path cost is a nondecreasing function of the depth of the node (e.g. all operators of equal cost) since nodes explored in depth order

Depth-first search

- Always expand node at deepest level of the tree, i.e. one of the most recently generated nodes. When hit a dead-end, backtrack to last choice



Iterative deepening

- Conduct a series of depth-limited searches, increasing depth-limit each time.
- Seems wasteful since work is repeated, but most work is at the leaves at each iteration and is not repeated.

Depth-first:

$$1 + b + b^2 + \ldots + b^{d-2} + b^{d-1} + b^d$$

Iterative deepening:

$$(d+1)1 + db + (d-1)b^2 + \ldots + 3b^{d-2} + 2b^{d-1} + 1b^d$$

Heuristic Search

- Heuristic or informed search exploits additional knowledge about the problem that helps direct search to more promising paths.

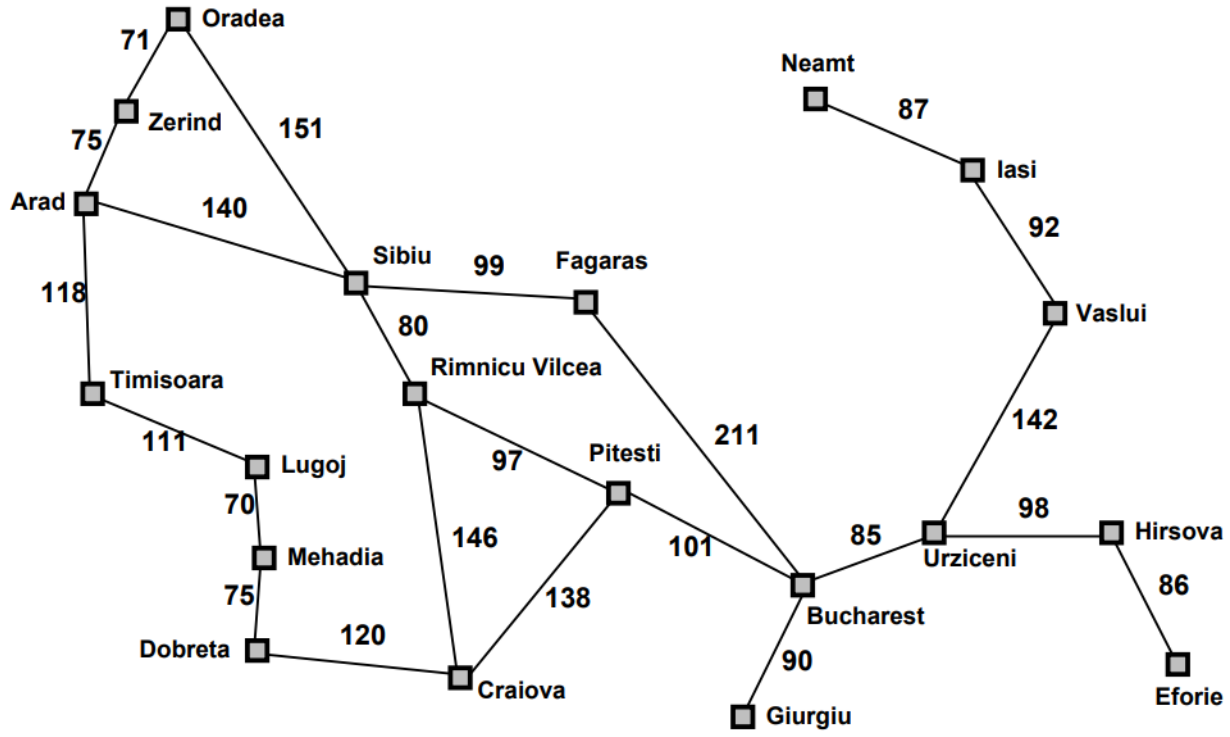
- A **heuristic function**, $h(n)$, provides an estimate of the cost of the path from a given node to the closest goal state

- Must be zero if node represents a goal state.
  - Example: Straight-line distance from current location to the goal location in a road navigation problem

- Many search problems are NP-complete so in the worst case still have exponential time complexity; however a good heuristic can:
  - Find a solution for an average problem efficiently.
  - Find a reasonably good but not optimal solution efficiently.

Best first search

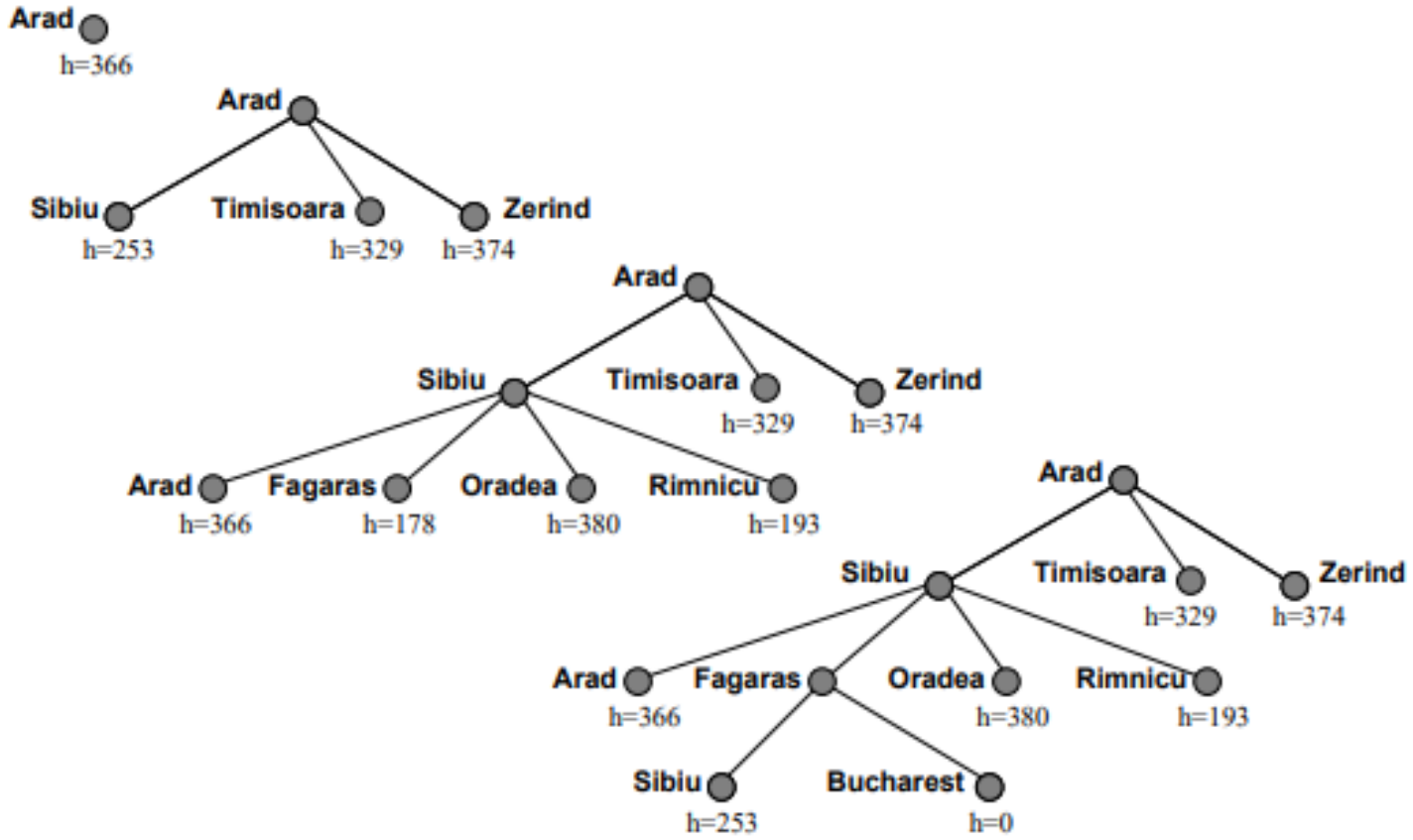
- At each step, best-first search sorts the queue according to a heuristic function



Straight–line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

AIMA Slides ©Stuart Russell and Peter Norvig, 1998

Best first search



- Does not find shortest path to goal (through Rimnicu) since it is only focused on the cost remaining rather than the total cost

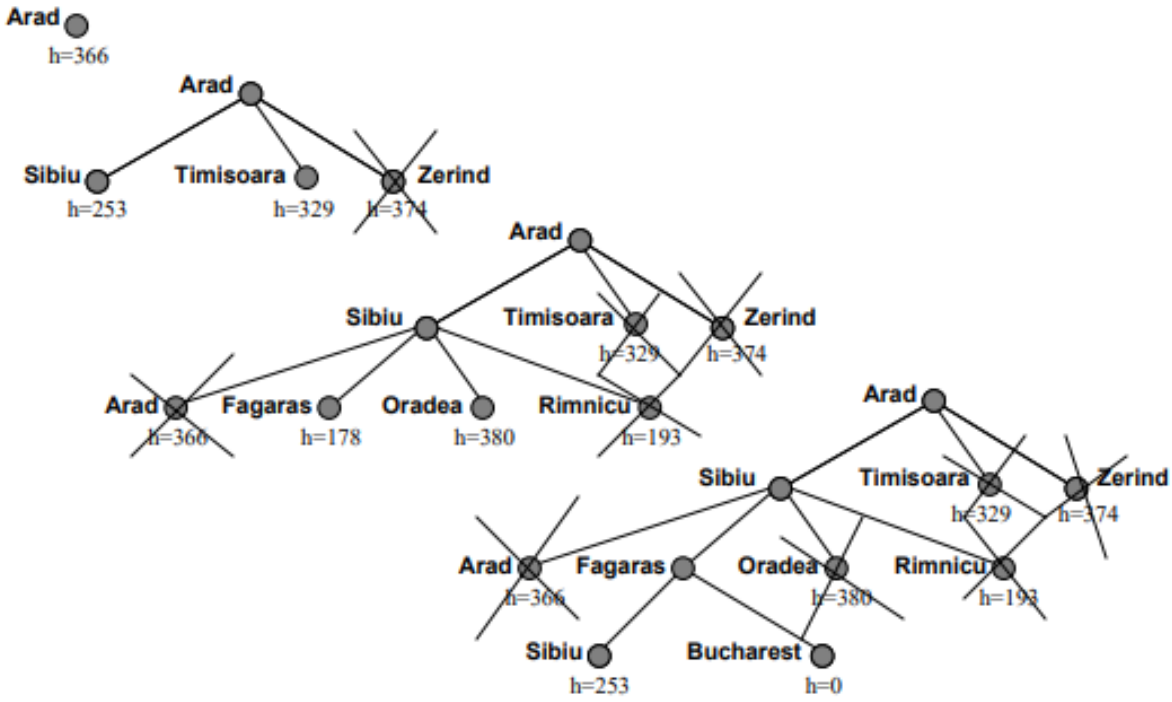
Best first search properties

- Not complete, may follow infinite path if heuristic rates each state on such a path as the best option. Most reasonable heuristics will not cause this problem however
- Worst case time complexity is still $O(b^m)$ where m is the maximum depth.
- Since must maintain a queue of all unexpanded states, space-complexity is also $O(b^m)$
- However, a good heuristic will avoid this worst-case behavior for most problems.

Beam search

- Space and time complexity of storing and sorting the complete queue can be too inefficient
- Beam search trims queue to the best *n* options (*n* is called the beam width) at each point
- Focuses search more but may eliminate solution even for finite search graphs

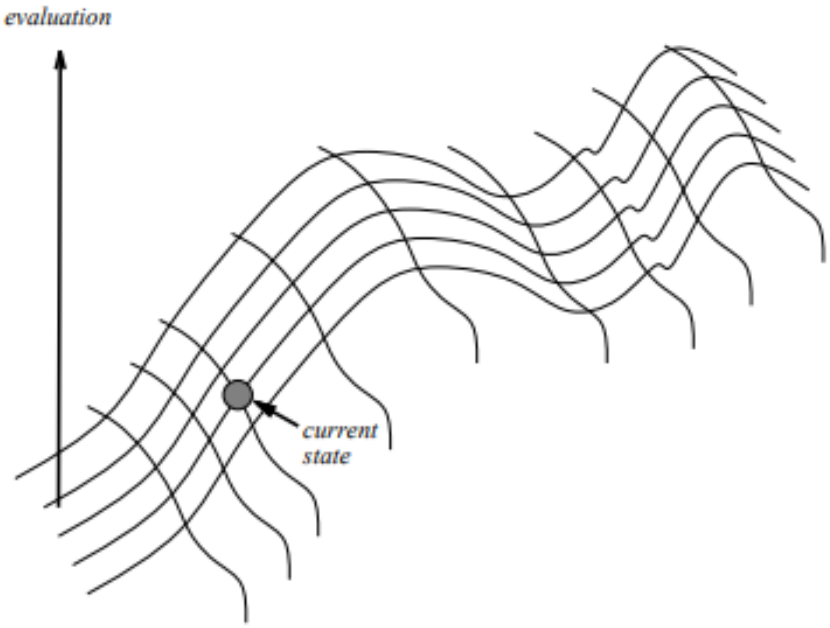
Example for n=2



AIMA Slides ©Stuart Russell and Peter Norvig, 1998

Hill climbing

- Beam search with a beam-width of 1 is called hill climbing
- Pursues locally best option at each point, i.e. the best successor to the current best node.
- Subject to local maxima, plateaux, and ridges.



AIMA Slides ©Stuart Russell and Peter Norvig, 1998

Comparing Hill Climbing, Beam, and Best-First

- **Hill climbing:** Keep one current solution and move to a better neighbor. Very low memory, but easily trapped in a local optimum.
- **Beam search:** Keep the best B partial solutions at each depth or planning stage. B=1 is close to greedy search; larger B preserves more alternatives.
- **Best-first search:** Maintain a priority frontier of discovered candidates and always expand the currently most promising one, even if it lies on a different branch or depth.
- **Shared dependence:** All three depend critically on the scoring function or heuristic. A misleading score can make the search systematically wrong.

Method	Candidates kept	How next state is chosen	Main advantage	Main weakness
Hill climbing	1	Best improving neighbor of current state	Minimal memory and computation	Can stop at local maximum; alternatives are forgotten
Beam search	B	Expand current beam and retain top B	Controls cost while preserving several hypotheses	Optimal branch can be permanently pruned
Best-first search	Potentially many	Best score anywhere in the frontier	Can switch among competing branches	Frontier can become large; quality depends on heuristic

Minimizing total cost: A* Search

- A* combines features of uniform cost search (complete, optimal, inefficient) with best-first (incomplete, non-optimal, efficient).
- Sort queue by estimated total cost of the completion of a path:

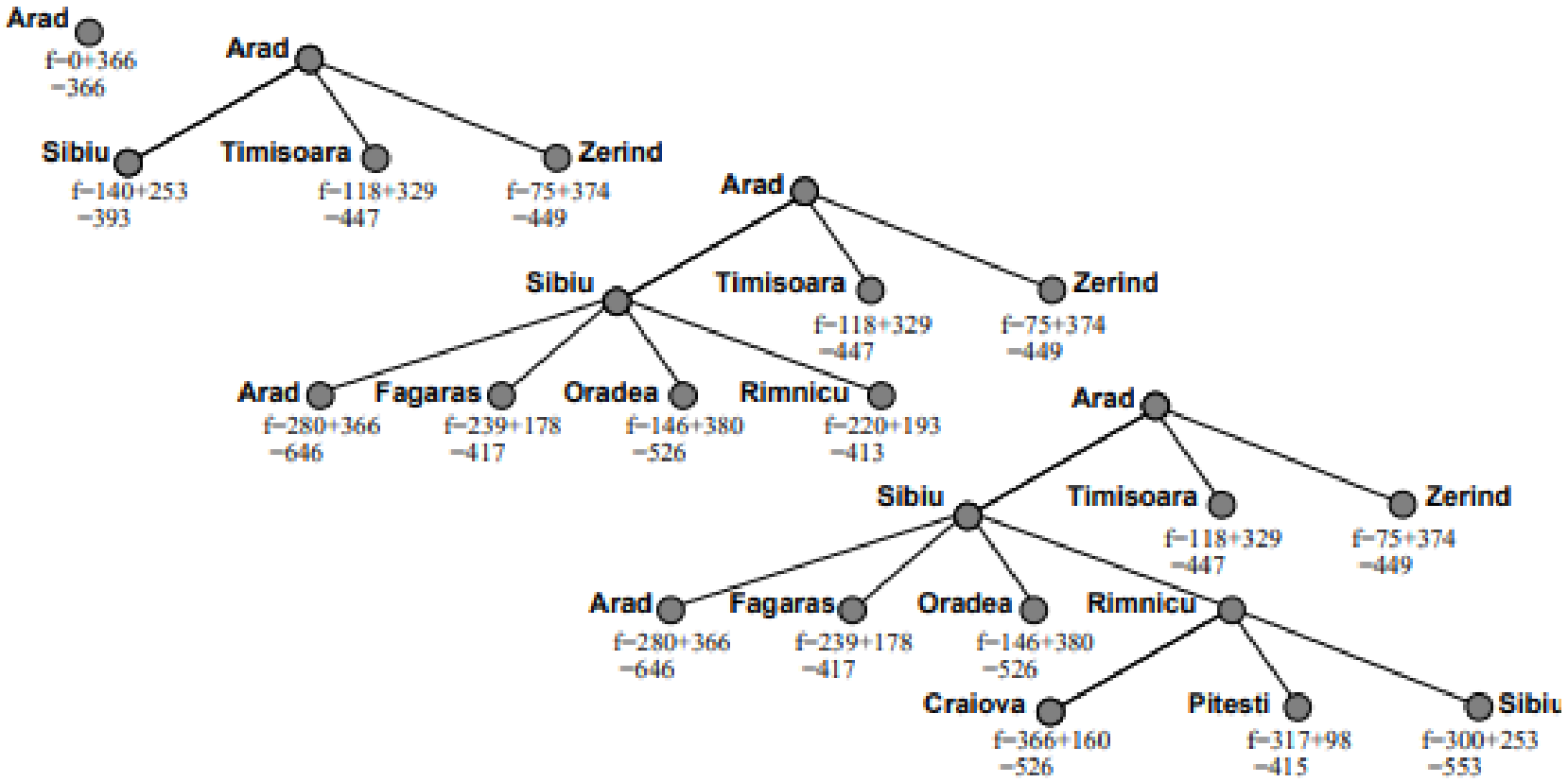
$$f(n) = g(n) + h(n)$$

Cost to this moment Estimated cost to the goal

- If the heuristic function always underestimates the distance to the goal, it is said to be admissible.
- If h is admissible, then $f(n)$ never overestimates the actual cost of the best solution through n

A* Search Example

AIMA Slides ©Stuart Russell and Peter Norvig, 1998

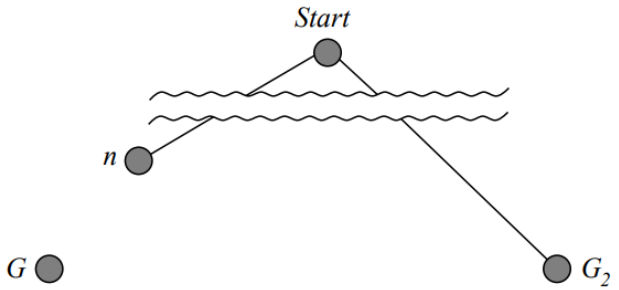


- Finds the optimal path to Bucharest through Rimnicu and Pitesti

Optimality of A*

If h is admissible, A* will always find a least cost path to the goal

- Proof by contradiction:
  - Let *G* be an optimal goal state with a path cost f*
  - Let G_2 be a suboptimal goal state supposedly found by A*
  - Let *n* be a current leaf node on an optimal path to *G*



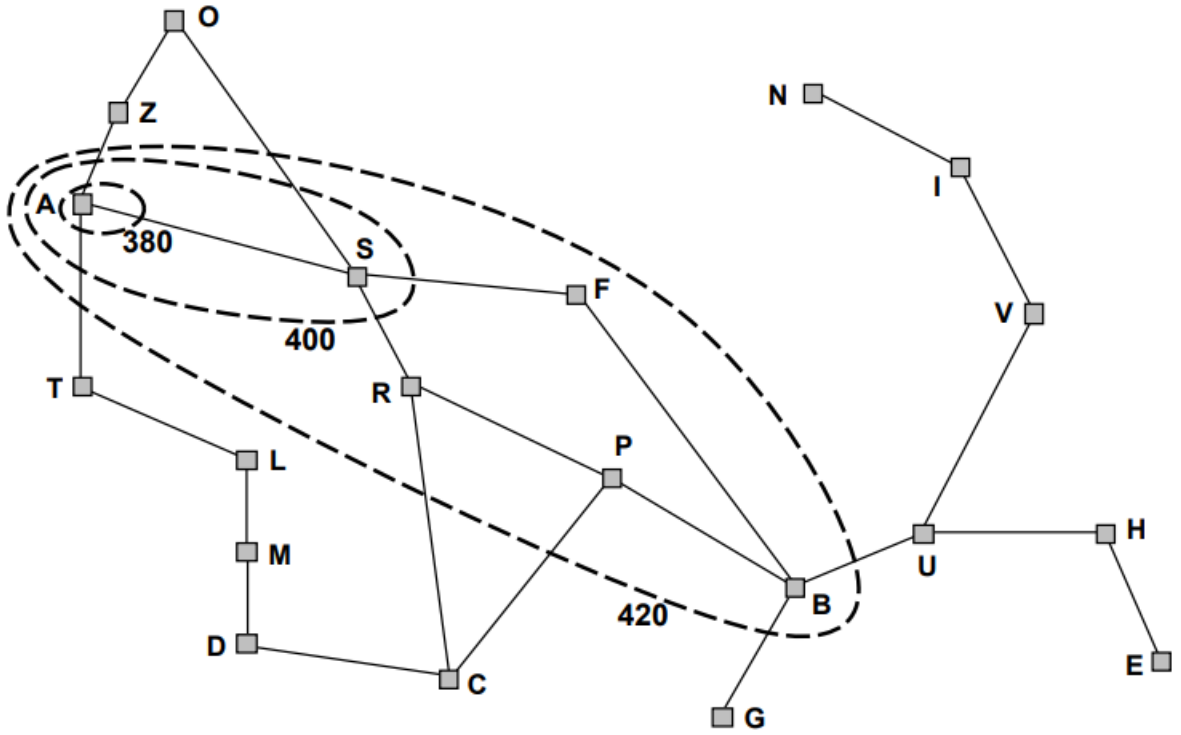
- Since h is admissible: f* >= f(*n*)
- If G2 is chosen for expansion over n then: f(*n*) >= f(G_2)
- Therefore: f* >= f(G_2)

- Since G_2 is a goal state, h(G_2)=0, therefore f(G2) = g(G_2) and f* >= g(G_2)

- Therefore G_2 is optimal. Contradiction.

Lemma: A* expands nodes in order of increasing f value

Gradually adds "f-contours" of nodes (cf. breadth-first adds layers)
Contour i has all nodes with $f = f_i$, where $f_i < f_{i+1}$



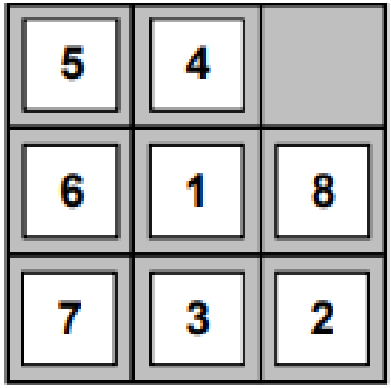
AIMA Slides ©Stuart Russell and Peter Norvig, 1998

Other properties of A*

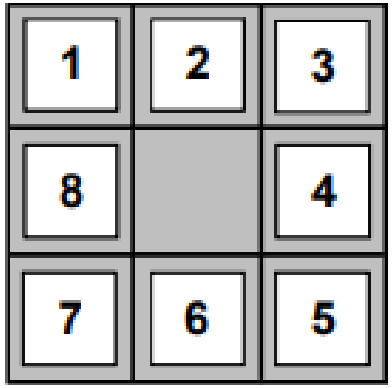
- A* is complete as long as
  - Branching factor is always finite
  - Every operator adds cost at least $\delta > 0$
- Time and space complexity still $O(b^m)$ in the worst case, since the search must maintain and sort complete queue of unexplored options.
- However, with a good heuristic can find optimal solutions for many problems in reasonable time
- Again, space complexity is a worse problem than time.

But what about heuristic functions?

- 8-puzzle search space
  - Typical solution length: 20 steps
  - Average branching factor: 3
  - Exhaustive search: $3^{20}=3.5 \times 10^9$
  - Bound on unique states:
    9! = 362,880



Start State



Goal State

- Admissible Heuristics:
  - Number of tiles out of place (h1): 7
  - City-block (Manhattan) distance (h2): 2+3+3+2+4+2+0+2=18

Using heuristics:

- Experiments on sample problems can determine the number of nodes searched and CPU time for different strategies

- One other useful measure is effective branching factor: If a method expands N nodes to find solution of depth d, and a uniform tree of depth d would require a branching factor of b* to contain N nodes, the effective branching factor is b*, so that N = 1 + b* + (b*)2 + ...+ (b*)d

Experimental Results on 8-puzzle problems

	Search Cost			Effective Branching Factor		
d	IDS	A*(h_1)	A*(h_2)	IDS	A*(h_1)	A*(h_2)
2	10	6	6	2.45	1.79	1.79
4	112	13	12	2.87	1.48	1.45
6	680	20	18	2.73	1.34	1.30
8	6384	39	25	2.80	1.33	1.24
10	47127	93	39	2.79	1.38	1.22
12	364404	227	73	2.78	1.42	1.24
14	3473941	539	113	2.83	1.44	1.23
16	–	1301	211	–	1.45	1.25
18	–	3056	363	–	1.46	1.26
20	–	7276	676	–	1.47	1.27
22	–	18094	1219	–	1.48	1.28
24	–	39135	1641	–	1.48	1.26

Quality of heuristics

- Since A* expands all nodes whose f value is less than that of an optimal solution, it is always better to use a heuristic with a higher value as long as it does not over-estimate.
- Therefore h_2 is uniformly better than h_1 or h_2 dominates h_1
- A heuristic should also be easy to compute, otherwise the overhead of computing the heuristic could outweigh the time saved by reducing search (e.g. using full breadth-first search to estimate distance wouldn't help).

Inventing Heuristics

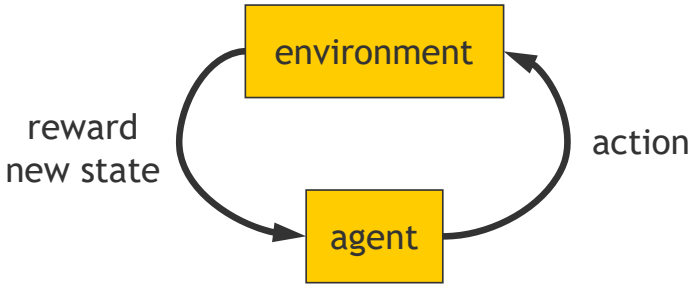
- Many good heuristics can be invented by considering relaxed versions of the problem (abstractions)

- **For 8-puzzle:** A tile can move from square A to B if A is adjacent to B and B is blank
  - (a) A tile can move from square A to B if A is adjacent to B
  - (b) A tile can move from square A to B if B is blank
  - (c) A tile can move from square A to B

- If there are a number of features that indicate a promising or unpromising state, a weighted sum of these features can be useful. Learning methods can be used to set weights.

Markov Decision Processes (MDP)

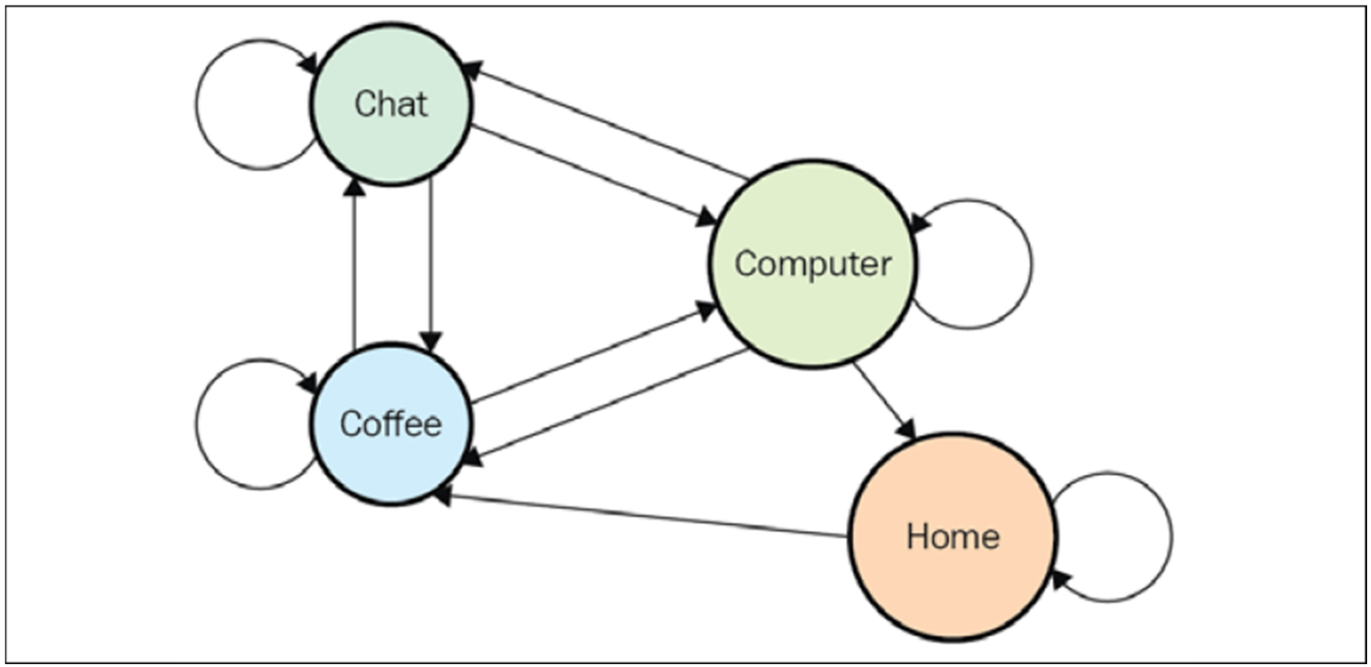
- Set of states S, set of actions A, initial state S_0
- Transition model $P(s,a,s')$
  - $P(\ [1,1], \text{up}, [1,2]\ ) = 0.8$
- Reward function $r(s)$
  - $r(\ [4,3]\ ) = +1$



- **Goal:** maximize cumulative reward in the long run
- **Policy:** mapping from S to A
  - $\pi(s)$ or $\pi(s,a)$ (deterministic vs. stochastic)
- **Reinforcement learning:**
  - transitions and rewards usually not available
  - how to change the policy based on experience
  - how to explore the environment

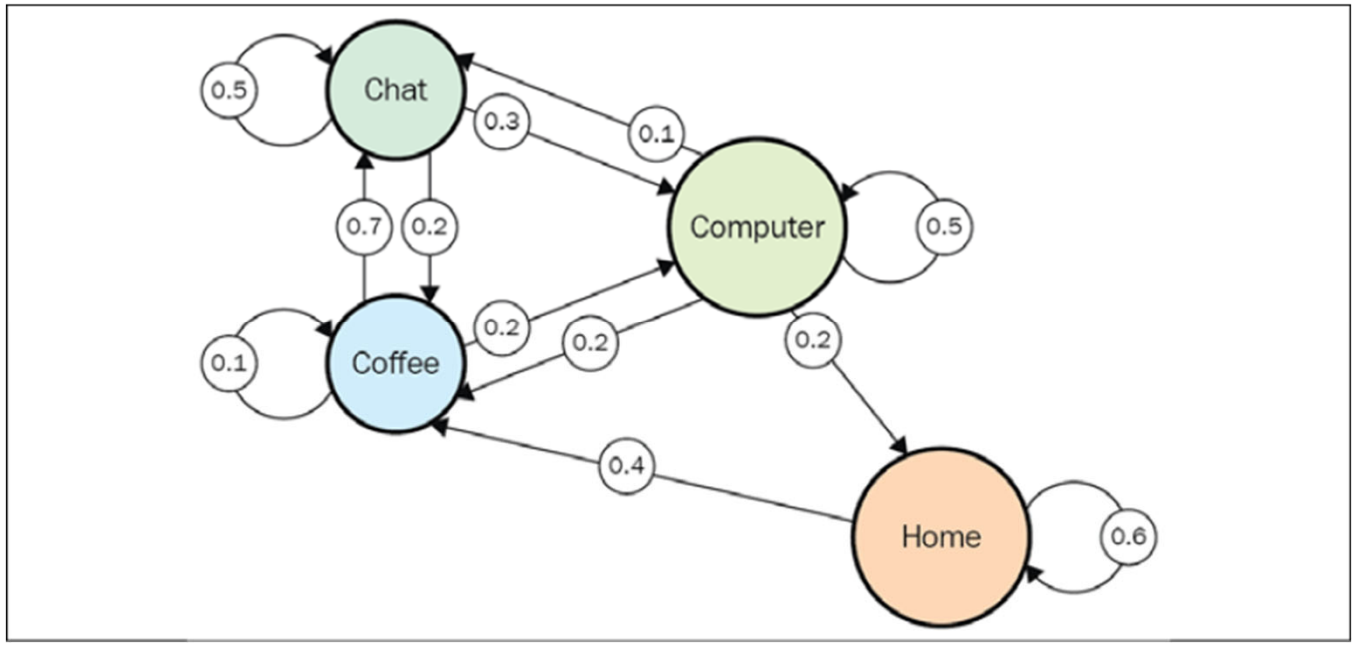
From Reinforcement Learning Tutorial,
Peter Bodík, RAD Lab, UC Berkeley

Let's define states



https://subscription.packtpub.com/book/data/9781838826994/1/ch01lvl1sec07/the-theoretical-foundations-of-rl

And transition probabilities



Finite Markov chain

An ***integer time stochastic process***, consisting of a ***domain D*** of $m>1$ states $\{s_1,\ldots,s_m\}$ and

1. An ***m*** dimensional ***initial distribution vector*** $(p(s_1),\ldots, p(s_m))$.
2. An ***m×m transition probabilities matrix M***= $(a_{s_i s_j})$

For example,

- ***D*** can be the letters $\{A, C, T, G\}$,
- $p(A)$ the probability of A to be the 1st letter in a sequence, and
- a_{AG} the probability that G follows A in a sequence.

Markov Chain



For each integer n, a Markov Chain assigns probability to sequences $(x_1 \ldots x_n)$ over $\boldsymbol{D}$ (i.e, $x_i \in \boldsymbol{D}$) as follows:

$$p((x_1, x_2, \ldots x_n)) = p(X_1 = x_1) \prod_{i=2}^{n} p(X_i = x_i \mid X_{i-1} = x_{i-1}) = p(x_1) \prod_{i=2}^{n} a_{x_{i-1} x_i}$$

Similarly, $(\boldsymbol{X_1}, \ldots, \boldsymbol{X_i}, \ldots)$ is a sequence of probability distributions over $\boldsymbol{D}$.

There is a rich theory which studies the properties of these sequences.

Markov Chain



Similarly, each $\boldsymbol{X_i}$ is a probability distributions over $\boldsymbol{D}$, which is determined by the initial distribution (p_1,..,p_n) and the transition matrix $\boldsymbol{M}$.

There is a rich theory which studies the properties of such "Markov sequences" ($\boldsymbol{X_1}$,..., $\boldsymbol{X_i}$,...).

Matrix Representation

	A	B	C	D
A	0.95	0	0.05	0
B	0.2	0.5	0	0.3
C	0	0.2	0	0.8
D	0	0	1	0

The transition probabilities Matrix $\boldsymbol{M} = (\boldsymbol{a_{st}})$

$\boldsymbol{M}$ is a stochastic Matrix:

$$\sum_t a_{st} = 1$$

The initial ***distribution vector*** $(u_1 \ldots u_m)$ defines the distribution of $\boldsymbol{X}_1$ $(p(\boldsymbol{X}_1 = s_i) = u_i)$.

Then after one move, the distribution is changed to $\boldsymbol{X_2} = \boldsymbol{X_1 M}$

Matrix Representation

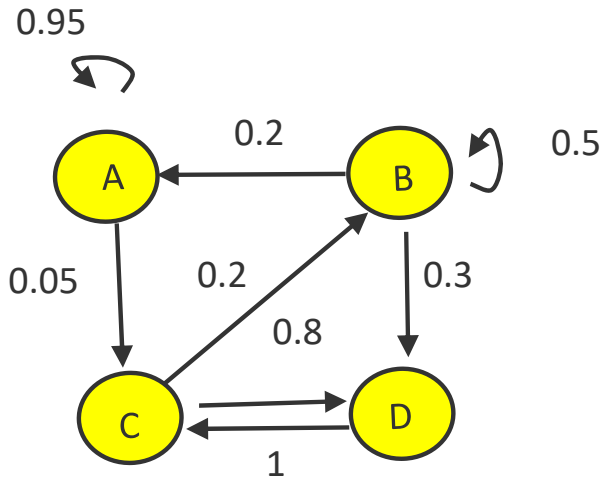
	A	B	C	D
A	0.95	0	0.05	0
B	0.2	0.5	0	0.3
C	0	0.2	0	0.8
D	0	0	1	0

Example: if $\mathbf{X_1}$=(0, 1, 0, 0)
then $\mathbf{X_2}$=(0.2, 0.5, 0, 0.3)

And if $\mathbf{X_1}$=(0, 0, 0.5, 0.5)
then $\mathbf{X_2}$=(0, 0.1, 0.5, 0.4).

The i-th distribution is $\boldsymbol{X_i} = \boldsymbol{X_1 M^{i-1}}$

Representation of Markov Chain as Digraph



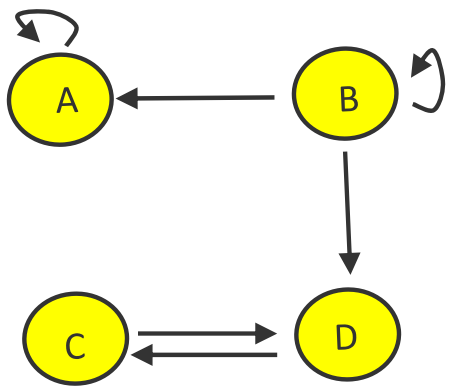
	A	B	C	D
A	0.95	0	0.05	0
B	0.2	0.5	0	0.3
C	0	0.2	0	0.8
D	0	0	1	0

Each directed edge A→B is associated with the **positive** transition probability from A to B.

Properties of Markov Chain States

States of Markov chains are classified by the digraph representation (omitting the actual probability values)

A, C and D are ***recurrent*** states: they are in strongly connected components which are **sinks** in the graph.

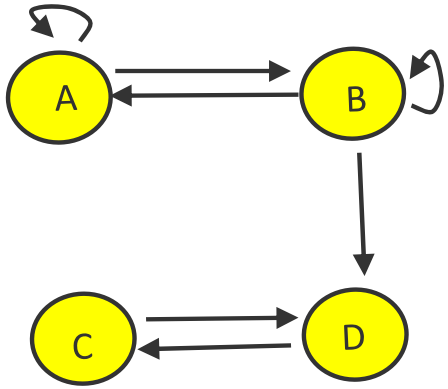


B is not recurrent – it is a ***transient*** state

Alternative definitions:

A state ***s*** is ***recurrent*** if it can be reached from any state reachable from ***s;***
otherwise it is ***transient***.

Another example: transient and recurrent states

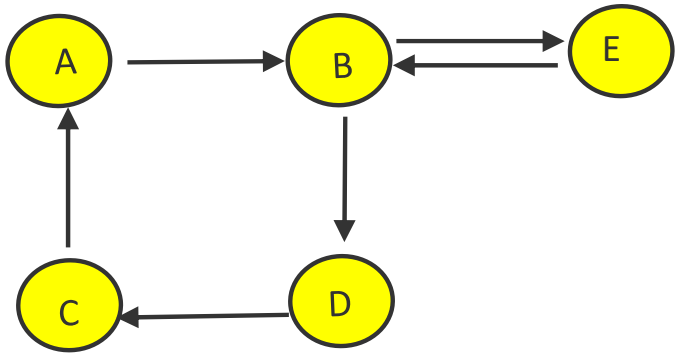


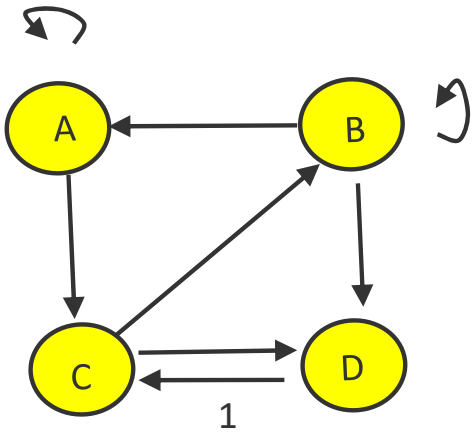
A and **B** are ***transient*** states**, C** and **D** are ***recurrent*** states.

Once the process moves from **B** to **D**, it will never come back.

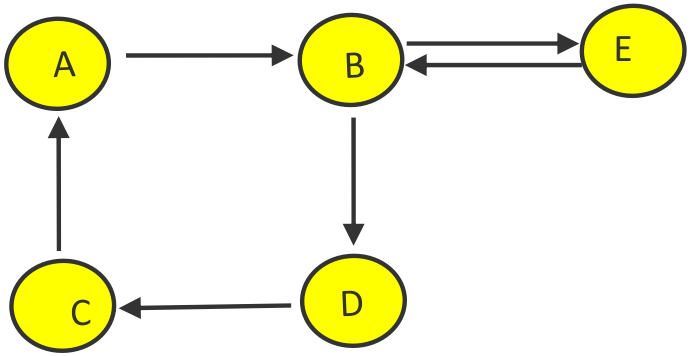
Irreducible Markov Chain

A Markov Chain is ***irreducible*** if the corresponding graph is strongly connected (and thus all its states are recurrent).





Periodic States



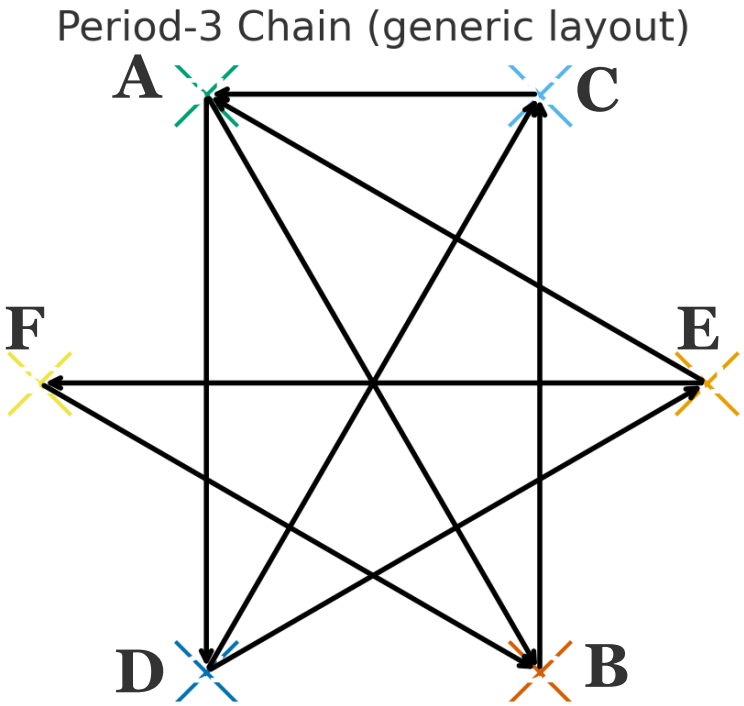
A state *s* has a period ***k*** if ***k*** is the ***GCD*** of the lengths of all the cycles that pass via *s*.

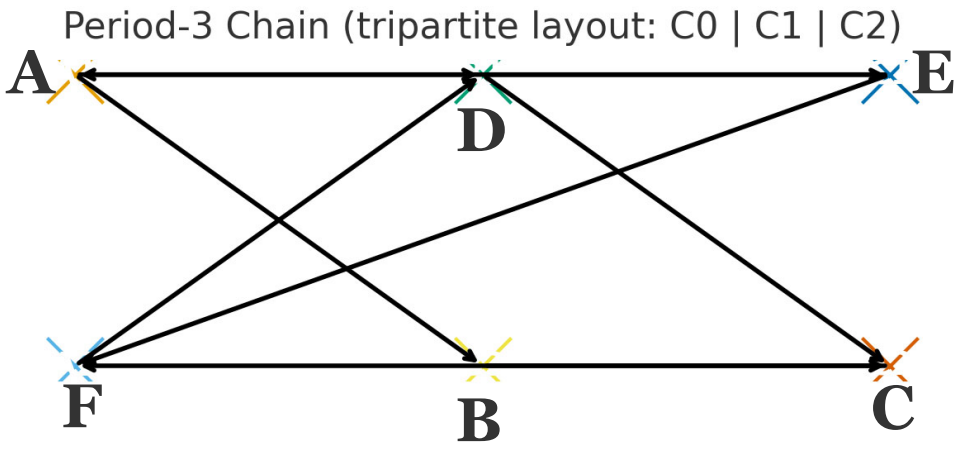
In the shown graph the period of A is 2.

All the states in the same strongly connected component have the same period

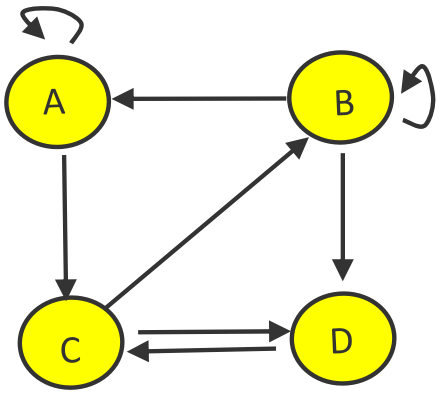
A Markov Chain is ***periodic*** if all the states in it have a period $k > 1$. It is ***aperiodic*** otherwise.

Periodic Chain: k = 3





Ergodic Markov Chain



A Markov chain is ***ergodic*** if :

1. ***the corresponding graph is strongly connected.***
2. ***It is not periodic***

Ergodic Markov Chains are important since they guarantee the corresponding Markovian process converges to a unique distribution, in which all states have strictly positive probability.

Stationary Distribution for Markov Chain

Let ***M*** be a Markov Chain of m states, and let $\boldsymbol{V} = (v_1,\ldots,v_m)$ be a probability distribution over the m states

$\boldsymbol{V} = (v_1,\ldots,v_m)$ is ***stationary distribution*** for ***M*** if $\boldsymbol{VM}=\boldsymbol{V}$.

(i.e., if one step of the process does not change the distribution).

V is a stationary distribution



V is a left (row) Eigenvector of ***M*** with Eigenvalue 1.

Good Markov Chain

A Markov Chains is *good* if the distributions X_i, as $I \to \infty$:

(1) converge to a unique distribution, independent of the initial distribution.

(2) In that unique distribution, each state has a positive probability.

The Fundamental Theorem of Finite Markov Chains:
A Markov Chain is good $\Leftrightarrow$ the corresponding graph is ergodic.